

Tutorial 13: EKF for Quadraped State Estimation

Athletic Intelligence in Robotics (AIR) Course

Mihaela Popescu

06.05.2026

Outline

- 1 Introduction
- 2 EKF Description
- 3 Tutorial Tasks
- 4 Conclusion

- 1 Introduction
- 2 EKF Description
- 3 Tutorial Tasks
- 4 Conclusion

Tutorial Goal

- Implement an EKF for quadruped state estimation.
- Fuse:
 - IMU measurements for prediction.
 - Stance-foot contacts for correction.
- Study the effect of:
 - IMU noise
 - IMU bias
 - foot slip
 - process and measurement noise tuning
 - missing contacts / flight phases

Main idea

IMU integration predicts motion, while stance feet provide temporary world-frame anchors that reduce drift.

- 1 Introduction
- 2 EKF Description**
- 3 Tutorial Tasks
- 4 Conclusion

State Representation in the EKF

- The EKF maintains an estimate of the system state:

$$\hat{x} = \left[R_{WB}, v_W, p_W, p_{F,W}^{(1)}, p_{F,W}^{(2)}, \dots \right]$$

- $R_{WB} \in \text{SO}(3)$: rotation from body frame $\{B\}$ to world frame $\{W\}$.
 - v_W : base velocity in world frame.
 - p_W : base position in world frame.
 - $p_{F,W}^{(i)}$: world-frame position of foot i during stance.
- The covariance is defined over an **error (perturbation) state** δx :

$$\delta x = \left[\delta\theta, \delta v, \delta p, \delta p_F^{(1)}, \delta p_F^{(2)}, \dots \right]$$

- **Error-state EKF**: the filter estimates a small correction δx and injects it into the state, ensuring consistent handling of orientation on $\text{SO}(3)$.

Contact-Dependent Foot States

- Foot states are added only when a foot enters contact.
- When a foot is in stance, the no-slip assumption gives:

$$p_{F,W} \approx \text{constant}$$

- When a new stance foot is detected, initialize its world position as:

$$p_{F,W} = p_W + R_{WB}p_{F,B}$$

- Here:
 - $p_{F,B}$ is the measured foot position in the body frame.
 - $p_{F,W}$ is the stored foot anchor in the world frame.
- When contact ends, the corresponding foot state is removed.

Note

A swing foot should not be treated as fixed, because it moves relative to the world.

Useful Operator: Skew Matrix

- The skew matrix represents the cross product:

$$[a]_{\times} b = a \times b$$

- For

$$a = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix},$$

the skew matrix is:

$$[a]_{\times} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}$$

- Small rotations are applied using the **matrix exponential**:

$$R_{\text{new}} = R \exp([\delta \theta]_{\times})$$

Prediction Step: Orientation

- The **gyroscope** measures angular velocity in the **body frame**:

$$\omega_B = [\omega_x \quad \omega_y \quad \omega_z]^T$$

- Over one time step Δt , compute:

$$\Delta R = \exp([\omega_B]_{\times} \Delta t)$$

- Propagate orientation:

$$R_{WB,k} = R_{WB,k-1} \Delta R$$

Prediction Step: Acceleration, Velocity, Position

- The **accelerometer** measures specific force in the **body frame**.
- Rotate it into the world frame and add gravity:

$$a_W = R_{WB,k} a_B + g_W$$

- Propagate velocity:

$$v_{W,k} = v_{W,k-1} + a_W \Delta t$$

- Propagate position:

$$p_{W,k} = p_{W,k-1} + v_{W,k-1} \Delta t + \frac{1}{2} a_W \Delta t^2$$

Prediction Step: Error-State Jacobian

- Covariance propagation:

$$P_k^- = F_k P_{k-1} F_k^T + Q$$

- Error-state vector**, where δp_F are the stacked errors of all active feet:

$$\delta x = [\delta \theta \quad \delta v \quad \delta p \quad \delta p_F]^T$$

- Process Jacobian** $F_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k-1}}$ used to propagate the error state $\delta x_k \approx F_k \delta x_{k-1}$:

$$F_k = \begin{bmatrix} F_{\theta\theta} & 0 & 0 & 0 \\ F_{v\theta} & I & 0 & 0 \\ F_{p\theta} & F_{pv} & I & 0 \\ 0 & 0 & 0 & I \end{bmatrix} \quad \begin{aligned} F_{\theta\theta} &= \Delta R^T \\ F_{p\theta} &= -\frac{1}{2} R_{WB,k} [a_B]_{\times} \Delta t^2 \end{aligned} \quad \begin{aligned} F_{v\theta} &= -R_{WB,k} [a_B]_{\times} \Delta t \\ F_{pv} &= I \Delta t \end{aligned}$$

- Process noise Q captures model uncertainty:

$$Q = \text{diag}(Q_{\theta}, Q_v, Q_p)$$

Contact Update: Measurement Model

- For a stance foot, predict the foot position from the current base estimate:

$$\hat{p}_{F,W} = p_W + R_{WB}p_{F,B}$$

- Compare this with the stored foot anchor:

$$r = \hat{p}_{F,W} - p_{F,W}$$

- If the foot is really fixed, this residual should be close to zero.
- The EKF update uses this residual to correct:
 - base orientation,
 - base position,
 - foot anchor position.

Key assumption

The stance foot does not slip.

Contact Update: Measurement Jacobian

- The residual is:

$$r = p_W + R_{WB}p_{F,B} - p_{F,W}$$

- **Measurement Jacobian** $H_k = \frac{\partial h}{\partial x} \big|_{\hat{x}_k^-}$ maps the error state to the residual $r_k \approx H_k \delta x_k$:

$$H = [H_\theta \quad H_v \quad H_p \quad H_{p_F}]$$

- Main blocks:

$$H_\theta = -R_{WB}[p_{F,B}]_\times, \quad H_v = 0, \quad H_p = I, \quad H_{p_F} = -I$$

- *Remark:* The orientation Jacobian H_θ can be unreliable in real world scenarios due to foot slip, imperfect contact detection, kinematic modeling errors, compliance and soft ground, frame misalignment, etc.
- For robustness, set $H_\theta = 0$ on real data. Position terms still constrain the base position:

$$H_p = I, \quad H_{p_F} = -I$$

Contact Update: Kalman Correction

- Innovation covariance:

$$S = HP^-H^T + R$$

- Kalman gain:

$$K = P^-H^TS^{-1}$$

- Error-state correction:

$$\delta x = -Kr$$

- Inject the correction:

$$R_{WB} \leftarrow R_{WB} \exp([\delta\theta]_{\times})$$

$$v_W \leftarrow v_W + \delta v$$

$$p_W \leftarrow p_W + \delta p$$

$$p_{F,W} \leftarrow p_{F,W} + \delta p_F$$

Contact Update: Covariance

- Use the Joseph-form covariance update:

$$P = (I - KH)P^-(I - KH)^T + KRK^T$$

- This is more numerically stable than:

$$P = (I - KH)P^-$$

- It helps preserve:

$$P = P^T, \quad P \succeq 0$$

- In code, symmetrize after the update:

$$P \leftarrow \frac{1}{2}(P + P^T)$$

What Can Go Wrong?

- **IMU noise:** causes random drift after integration.
- **IMU bias:** causes systematic drift, often much worse than noise.
- **No contact:** the robot is temporarily dead-reckoning from IMU only.
- **Foot slip:** violates the fixed-contact assumption.
- **Wrong noise tuning:**
 - small Q : filter trusts prediction too much
 - large R : filter ignores foot updates
 - small R : filter over-trusts possibly slipping contacts

- 1 Introduction
- 2 EKF Description
- 3 Tutorial Tasks**
- 4 Conclusion

Tutorial Workflow

- 1 Run the provided simulation and inspect the data.
- 2 Visualize the EKF state and active foot states.
- 3 Implement IMU prediction.
- 4 Test IMU-only estimation.
- 5 Add IMU noise and bias.
- 6 Implement foot-contact correction.
- 7 Add foot slip and observe failure cases.
- 8 Tune Q and R .
- 9 Visualize covariance ellipses.
- 10 Apply the same estimator to real quadruped data.

Task 1–2: Data and State Inspection

- Start with synthetic data. This provides ground truth for evaluation as follows:

$$e_p(t) = p_{\text{est}}(t) - p_{\text{gt}}(t)$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N \|e_p(k)\|^2}$$

- Inspect the initialized state:

$$x = [R_{WB}, v_W, p_W]$$

- Then add stance-foot states using contact information:

$$x = [R_{WB}, v_W, p_W, p_{F,W}^{(1)}, \dots]$$

Task 3: Implement IMU Propagation

- ① Propagate orientation:

$$R_k = R_{k-1} \exp([\omega_k]_{\times} \Delta t)$$

- ② Compute world-frame acceleration:

$$a_W = R_k a_B + g_W$$

- ③ Integrate velocity:

$$v_k = v_{k-1} + a_W \Delta t$$

- ④ Integrate position:

$$p_k = p_{k-1} + v_{k-1} \Delta t + \frac{1}{2} a_W \Delta t^2$$

- ⑤ Build F_k and propagate:

$$P_k^- = F_k P_{k-1} F_k^T + Q$$

Task 4–5: IMU Noise and Bias

- Add random IMU noise (zero-mean Gaussian noise w_a, w_ω):

$$a_m = a + w_a, \quad \omega_m = \omega + w_\omega$$

- Add systematic IMU bias (b_a, b_ω):

$$a_m = a + b_a + w_a$$

$$\omega_m = \omega + b_\omega + w_\omega$$

Task 4–5: IMU Noise and Bias

- Add random IMU noise (zero-mean Gaussian noise w_a, w_ω):

$$a_m = a + w_a, \quad \omega_m = \omega + w_\omega$$

- Add systematic IMU bias (b_a, b_ω):

$$a_m = a + b_a + w_a$$

$$\omega_m = \omega + b_\omega + w_\omega$$

Answers

- Noise causes drift because acceleration and angular velocity are integrated.
- Bias causes larger long-term drift due to accumulation over time.
- Gyro bias is especially harmful:

$$\omega_m = \omega + b_\omega \Rightarrow R_{WB} \text{ drifts over time} \Rightarrow a_W \text{ is rotated incorrectly} \Rightarrow p_W \text{ drifts}$$

Task 6: Implement Foot-Contact Correction

- ① Compute predicted foot position:

$$\hat{p}_{F,W} = p_W + R_{WB}p_{F,B}$$

- ② Compute residual:

$$r = \hat{p}_{F,W} - p_{F,W}$$

- ③ Build measurement Jacobian:

$$H = \begin{bmatrix} -R_{WB}[p_{F,B}]_{\times} & 0 & I & -I \end{bmatrix}$$

- ④ Compute:

$$S = HP^{-1}H^T + R, \quad K = P^{-1}H^TS^{-1}$$

- ⑤ Correct state:

$$\delta x = -Kr$$

- ⑥ Update covariance using Joseph form.

Task 7: Foot Slip

- Introduce foot slip in the simulation.

Task 7: Foot Slip

- Introduce foot slip in the simulation.

Answer

- The contact update assumes $p_{F,W} \approx \text{constant}$.
- Foot slip violates this assumption:

$$p_{F,W}(t + \Delta t) \neq p_{F,W}(t)$$

- Then the residual is no longer caused only by state-estimation error.
- The EKF may apply a wrong correction:

$$\delta x = -Kr$$

- Result:
 - position error can increase,
 - orientation can be pulled in the wrong direction,
 - covariance can become overconfident,
 - the estimate can become inconsistent.

Task 8: Tuning Q and R

- Process noise Q controls trust in prediction:

$$P^- = FPF^T + Q$$

- Measurement noise R controls trust in contact updates:

$$S = HP^-H^T + R$$

Task 8: Tuning Q and R

- Process noise Q controls trust in prediction:

$$P^- = FPF^T + Q$$

- Measurement noise R controls trust in contact updates:

$$S = HP^-H^T + R$$

Answer

- If Q is too small:
 - the filter trusts the IMU prediction too much,
 - covariance becomes too small,
 - contact corrections have too little effect.
- If R is too large:
 - the filter ignores foot contacts,
 - behavior approaches IMU-only dead reckoning.
- If R is too small:
 - the filter over-trusts contacts,
 - foot slip can strongly corrupt the estimate.

Task 9: Covariance Ellipses

- The position covariance in the x - y plane is:

$$P_{xy} = \begin{bmatrix} P_{xx} & P_{xy} \\ P_{yx} & P_{yy} \end{bmatrix}$$

- Covariance ellipses visualize uncertainty.

Task 9: Covariance Ellipses

- The position covariance in the x - y plane is:

$$P_{xy} = \begin{bmatrix} P_{xx} & P_{xy} \\ P_{yx} & P_{yy} \end{bmatrix}$$

- Covariance ellipses visualize uncertainty.

Answer

- Ellipse size $\sim$ uncertainty (larger = higher, smaller = lower)
- IMU-only prediction - P grows; Contact updates - P shrinks.
- During flight phases, no foot anchor is available, so uncertainty grows.

Task 10: Real Data

- Apply the same EKF for real world data and tune the Q and R noise parameters.

Task 10: Real Data

- Apply the same EKF for real world data and tune the Q and R noise parameters.

Answer

- Real data is harder than simulation because of:
 - IMU bias and calibration errors,
 - time synchronization errors,
 - imperfect contact detection,
 - compliance and impacts,
 - kinematic errors,
 - foot slip,
 - uneven terrain.
- For rough terrain, possible improvements include:
 - contact probability estimation,
 - slip detection,
 - IMU bias estimation,
 - adaptive measurement noise,
 - visual or LiDAR odometry fusion.

- 1 Introduction
- 2 EKF Description
- 3 Tutorial Tasks
- 4 Conclusion**

Key Takeaways

- EKF combines:
 - IMU-based prediction (drift-prone),
 - contact-based corrections (drift-reducing).
- Stance feet act as temporary world-frame anchors.
- Performance depends strongly on:
 - noise tuning (Q , R),
 - contact quality (slip, detection),
 - modeling assumptions.
- Real-world data is more challenging than simulation.
- Error-state formulation enables efficient handling of orientation in $SO(3)$.